

Personal Loan Ledger Mobile Application

Database Design Specification

Shared Loan Tracking, Reminders, Repayment Confirmation, Dashboard Analytics, and Bank Profile Sharing

Document	Database Design Specification
Version	Version 1.0 - Product Foundation
Date	September 2026
Purpose	Defines the relational schema, data types, keys, constraints, indexes, encryption boundaries, transaction rules, and representative queries for PostgreSQL.

Product scope note: The application tracks peer-to-peer loans and repayment acknowledgements. It does not custody funds, execute bank transfers, provide credit scoring, or automatically enforce legal debt collection in the initial release.

Table of Contents

1. Database Design Goals
2. Conventions and Data Types
3. Entity Relationship Overview
4. Users and Profiles
5. User Sessions and Verification
6. Friendships and Blocks
7. Loans
8. Loan Terms and Amendments
9. Repayments
10. Bank Profiles
11. Bank Profile Shares
12. Notifications and Device Tokens
13. Loan Events / Audit Log
14. Idempotency Records
15. Attachments and Object References
16. Dashboard Query Model
17. Integrity Constraints
18. Indexes and Query Performance
19. Transactions and Locking
20. Encryption and Sensitive Data
21. Soft Delete, Retention, and Archival
22. Migration Strategy
23. Backup and Recovery
24. Sample SQL / DDL
25. Representative Queries
26. Data Reconciliation and Auditing
27. Data Dictionary Summary
28. Future Schema Extensions

1. Database Design Goals

The database is designed for shared financial-record integrity. The primary goals are transactional correctness, two-party traceability, precise money arithmetic, secure bank-profile handling, and efficient dashboard queries.

- PostgreSQL is recommended as the authoritative relational database.
- Core entities use explicit foreign keys and constraints rather than free-form JSON.
- Monetary values use NUMERIC/DECIMAL, never FLOAT/DOUBLE.
- Historical loan events are append-only.
- Deletion rules preserve counterpart financial history while minimizing unnecessary personal data.

2. Conventions and Data Types

Concept	Convention
Primary key	UUID (application or DB generated).
Timestamp	TIMESTAMPTZ stored in UTC.
Money	NUMERIC(20,4) or stricter scale based on supported currencies/product policy.
Currency	CHAR(3) ISO 4217 code where possible.
Status	PostgreSQL enum or validated text/check constraint; migrations must be planned carefully.
Boolean	BOOLEAN with explicit default.
Names	snake_case tables/columns; plural table names in this design.
Audit timestamps	created_at, updated_at; immutable/event tables may omit updated_at.
Soft delete	deleted_at only where preserving references is important.
Sensitive encrypted fields	BYTEA/TEXT ciphertext with separate masked/last4 helper fields.

3. Entity Relationship Overview

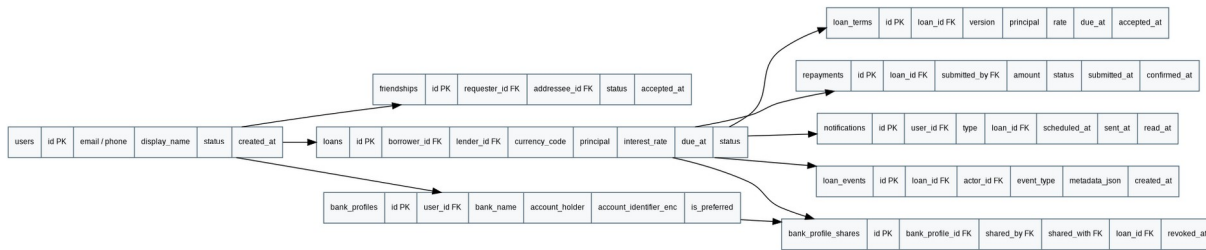


Figure 1. Simplified relational entity model.

- users participate in friendships and loans in borrower/lender roles.
- loans have versioned terms and one or more repayment attempts.
- bank_profiles belong to users and may be explicitly shared with another user and/or loan.
- loan_events form the immutable audit timeline.
- notifications and device_tokens support in-app and push delivery.

4. Users and Profiles

Column	Type	Constraints / Notes
id	uuid	PK
email	citext/varchar	Nullable if phone-only; unique when present.
phone_e164	varchar(20)	Nullable if email-only; unique when present.
username	citext/varchar	Optional unique public handle.
display_name	varchar(120)	Required after onboarding.
avatar_object_key	varchar	Nullable object storage reference.
timezone	varchar(64)	IANA timezone.
locale	varchar(16)	e.g., en, am-ET.
default_currency_code	char(3)	Nullable convenience setting.
status	varchar/enum	active, suspended, deletion_pending, deleted.
created_at	timestamptz	Required.
updated_at	timestamptz	Required.
deleted_at	timestamptz	Nullable soft deletion timestamp.

Identity retention: Historical loans should reference stable user IDs. If a user is deleted, personally identifying profile fields may be cleared/pseudonymized while the user row remains as a referential anchor where policy permits.

5. User Sessions and Verification

Table	Important Columns
user_sessions	id, user_id, refresh_token_hash, device_label, ip_hash optional, created_at, last_used_at, expires_at, revoked_at
verification_challenges	id, user_id/identifier, channel, code_hash, attempts, expires_at, consumed_at
password_reset_tokens	id, user_id, token_hash, expires_at, consumed_at

- Never store plaintext passwords, verification codes, refresh tokens, or reset tokens.
- Session lookup indexes should support user_id + revoked_at and token/session identifier.

6. Friendships and Blocks

Column	Type	Notes
id	uuid	PK
requester_id	uuid	FK users
addressee_id	uuid	FK users
user_low_id	uuid	Canonical lower UUID/string for uniqueness
user_high_id	uuid	Canonical higher counterpart
status	enum/text	pending, accepted, rejected, removed, blocked
requested_at	timestampz	Required
accepted_at	timestampz	Nullable
removed_at	timestampz	Nullable
blocked_by_user_id	uuid	Nullable; identifies blocker

- UNIQUE(user_low_id,user_high_id) prevents duplicate relationship rows.
- CHECK(requester_id <> addressee_id).
- Do not cascade-delete friendship rows when a user has financial history.

7. Loans

Column	Type	Notes
id	uuid	PK
reference_code	varchar(24)	Unique human-readable reference.
borrower_id	uuid	FK users; not null
lender_id	uuid	FK users; not null
currency_code	char(3)	Locked after activation
principal_amount	numeric(20,4)	Accepted principal; > 0
interest_type	varchar	none, flat_percent, fixed_amount; MVP uses none/flat_percent
interest_rate	numeric(9,4)	Nullable/0 allowed
interest_amount	numeric(20,4)	Server-calculated accepted interest snapshot
expected_total	numeric(20,4)	Principal + interest for accepted terms
confirmed_repaid_amount	numeric(20,4)	Optional denormalized value; reconciled from repayments
outstanding_amount	numeric(20,4)	Optional denormalized value; >=0
due_at	timestampz	Accepted due date
status	varchar/enum	Lifecycle state
accepted_at	timestampz	Nullable until active
completed_at	timestampz	Nullable
created_at	timestampz	Required
updated_at	timestampz	Required
row_version	bigint	Optional optimistic concurrency counter

- CHECK(borrower_id <> lender_id).
- CHECK(principal_amount IS NULL OR principal_amount > 0).
- After ACTIVE, application logic treats accepted financial fields as immutable; DB triggers are optional but application tests are required.

8. Loan Terms and Amendments

Column	Type	Notes
id	uuid	PK
loan_id	uuid	FK loans
version	integer	Starts at 1; unique per loan
proposed_by_user_id	uuid	FK users
principal_amount	numeric(20,4)	Terms snapshot
currency_code	char(3)	Terms snapshot
interest_type	varchar	Terms snapshot
interest_rate	numeric(9,4)	Terms snapshot
interest_amount	numeric(20,4)	Calculated snapshot
expected_total	numeric(20,4)	Calculated snapshot
due_at	timestamptz	Terms snapshot
note	text	User-provided; moderation/privacy rules apply
status	varchar	proposed, accepted, superseded, rejected
accepted_at	timestamptz	Nullable
created_at	timestamptz	Required

- `UNIQUE(loan_id,version)`.
- Exactly one accepted version should represent current accepted terms; enforce with partial unique index where practical.
- Amendments create a new version rather than updating old terms.

9. Repayments

Column	Type	Notes
id	uuid	PK
loan_id	uuid	FK loans
submitted_by_user_id	uuid	FK users; expected borrower
amount	numeric(20,4)	>0
status	varchar	pending, confirmed, rejected, cancelled
note	text	Optional
proof_attachment_id	uuid	Nullable
submitted_at	timestampz	Required
confirmed_by_user_id	uuid	Nullable; expected lender
confirmed_at	timestampz	Nullable
rejected_at	timestampz	Nullable
rejection_reason	text	Nullable

- CHECK(amount > 0).
- Application transaction prevents confirmed repayments exceeding outstanding balance.
- Indexes on (loan_id,status) and (confirmed_by_user_id,status) support detail/action queues.

10. Bank Profiles

Column	Type	Notes
id	uuid	PK
user_id	uuid	Owner FK
profile_type	varchar	bank_account, mobile_wallet, other
label	varchar(80)	e.g., Main bank
provider_name	varchar(120)	Bank/wallet name
account_holder_name	varchar(160)	May be sensitive PII
account_identifier_enc	bytea/text	Encrypted full account number / wallet ID
account_identifier_last4	varchar(8)	Masked display helper; optional
routing_data_enc	bytea/text	Optional encrypted routing/IBAN/SWIFT data
country_code	char(2)	Optional
currency_code	char(3)	Optional hint, not a loan constraint
is_preferred	boolean	At most one preferred per user via partial unique index
status	varchar	active, archived
created_at	timestamptz	Required
updated_at	timestamptz	Required

Encryption: The database should store ciphertext, not plaintext account identifiers. Key material is managed outside the database using KMS/secret management and supports rotation.

11. Bank Profile Shares

Column	Type	Notes
id	uuid	PK
bank_profile_id	uuid	FK
shared_by_user_id	uuid	Owner FK
shared_with_user_id	uuid	Recipient FK
loan_id	uuid	Nullable scope FK; recommended for repayment context
purpose	varchar	e.g., loan_repayment
created_at	timestampz	Required
revoked_at	timestampz	Nullable
last_revealed_at	timestampz	Nullable/audit convenience

- CHECK(shared_by_user_id <> shared_with_user_id).
- Share query requires revoked_at IS NULL and verifies owner/recipient relation to loan.
- Unique active share can be enforced with a partial unique index on profile+recipient+loan WHERE revoked_at IS NULL.

12. Notifications and Device Tokens

Table	Columns / Purpose
device_tokens	id, user_id, platform, token, device_id/hash, enabled, last_seen_at, created_at; unique active token.
notifications	id, user_id, type, loan_id nullable, title_key, body_key, safe_params_json, scheduled_at, sent_at, read_at, delivery_status, provider_message_id nullable.
notification_preferences	user_id, push_enabled, due_7d, due_3d, due_1d, due_today, overdue_cadence, quiet_hours/timezone settings.

- Do not store full bank data or unnecessarily sensitive free text in notification params.
- Indexes: notifications(user_id,read_at,created_at desc) and notifications(scheduled_at,delivery_status).

13. Loan Events / Audit Log

Column	Type	Notes
id	uuid	PK
loan_id	uuid	FK
actor_user_id	uuid	Nullable for system events
actor_admin_id	uuid	Nullable for admin event
event_type	varchar	Immutable semantic type
metadata_json	jsonb	Safe structured details only
created_at	timestampz	Server timestamp
correlation_id	uuid/varchar	Traceability across request/job

- No UPDATE/DELETE through normal application repository.
- Index (loan_id,created_at,id).
- System-generated events use actor_user_id NULL plus metadata/source.
- Sensitive bank plaintext is prohibited in metadata_json.

14. Idempotency Records

Column	Type	Notes
id	uuid	PK
user_id	uuid	FK
idempotency_key	varchar(100)	Client-generated key
operation	varchar(120)	Route/action identifier
request_hash	varchar	Detect same key/different payload
response_status	integer	Stored result
response_body_json	jsonb	Safe response snapshot where practical
created_at	timestamptz	Required
expires_at	timestamptz	Cleanup horizon

- UNIQUE(user_id,operation,idempotency_key).
- Do not store secret request fields in response snapshots.

15. Attachments and Object References

Column	Type	Notes
id	uuid	PK
owner_user_id	uuid	FK
object_key	varchar	Random storage path/key
original_filename	varchar	Sanitized; optional
mime_type	varchar	Validated
size_bytes	bigint	Limit enforced
purpose	varchar	avatar, repayment_proof, other
status	varchar	pending, ready, rejected, deleted
created_at	timestampz	Required
deleted_at	timestampz	Nullable

16. Dashboard Query Model

- For MVP, dashboard can be generated from loans plus confirmed repayments with indexes.
- If `outstanding_amount` is maintained on loans, treat it as denormalized but transactionally updated and periodically reconciled.
- Friend balances are grouped by counterparty and currency.
- No table stores a cross-currency “net worth” value in MVP.

View / Query	Purpose
<code>v_open_loan_balances</code>	Open loans with borrower/lender, currency, due date, outstanding.
<code>v_user_currency_positions</code>	Receivable/payable/net per user/currency.
<code>v_user_due_soon</code>	Open loans inside configured time window.
<code>v_user_pending_actions</code>	Incoming requests and repayment confirmations.

17. Integrity Constraints

Invariant	Mechanism
User cannot lend to self	CHECK borrower_id <> lender_id.
Friendship pair unique	Canonical pair unique constraint.
Loan reference unique	UNIQUE reference_code.
Terms version unique	UNIQUE(loan_id,version).
Preferred bank profile unique	Partial unique index user_id WHERE is_preferred AND status=active.
Positive money values	CHECK principal/repayment > 0.
No negative outstanding	CHECK outstanding_amount >= 0 if stored.
Bank share parties differ	CHECK shared_by <> shared_with.
Idempotency uniqueness	UNIQUE user+operation+key.
Foreign-key history safety	Use RESTRICT/soft-delete instead of destructive cascade for shared financial entities.

18. Indexes and Query Performance

Index	Purpose
loans(lender_id,status,currency_code,due_at)	Receivable dashboard and due list.
loans(borrower_id,status,currency_code,due_at)	Payable dashboard and due list.
loans(reference_code)	Direct lookup/support.
repayments(loan_id,status,submitted_at)	Loan detail and outstanding calculation.
friendships(user_low_id,user_high_id)	Pair lookup and uniqueness.
loan_events(loan_id,created_at,id)	Timeline pagination.
bank_profile_shares(shared_with_user_id,loan_id,revoked_at)	Authorized profile retrieval.
notifications(user_id,read_at,created_at desc)	Notification inbox.
device_tokens(user_id,enabled)	Push fanout.

- Use EXPLAIN ANALYZE against representative data before adding redundant indexes.
- Remember indexes increase write cost and storage; prioritize query-backed indexes.

19. Transactions and Locking

- Loan acceptance transaction: lock/validate requested loan, create accepted term version, update loan fields/status, append event, create notification intents, commit.
- Repayment confirmation transaction: lock repayment + loan, verify lender/state, mark repayment confirmed, recalculate balance, close loan if zero, append events/notifications, commit.
- Bank preferred switch transaction: clear old preferred and set new preferred atomically.
- Use SERIALIZABLE only when necessary; normally READ COMMITTED + explicit row locking/conditional updates is adequate and cheaper.

20. Encryption and Sensitive Data

- Database-level storage encryption is baseline but not sufficient for bank identifiers; use application field-level encryption too.
- Encryption keys live outside PostgreSQL.
- Store a key version with ciphertext if rotation strategy requires it.
- Search on encrypted account numbers should not be a product requirement; if duplicate detection is needed, use a carefully designed keyed hash/HMAC, not reversible plaintext indexes.
- Database dumps/backups remain sensitive and must be encrypted and access-controlled.

21. Soft Delete, Retention, and Archival

- Shared loan records cannot simply cascade-delete when one user deletes an account.
- Pseudonymize or null nonessential profile fields while retaining stable IDs and required financial history according to policy/legal review.
- Archived bank profiles should not be shareable; active shares should be revoked when appropriate.
- Event logs may have longer retention than operational notification records.
- Define purge jobs for expired idempotency rows, old verification challenges, invalid device tokens, and deleted attachments.

22. Migration Strategy

- Every schema change is represented as a versioned migration committed with application code.
- Production migrations are forward-compatible with rolling deploys where possible: add nullable column -> deploy code -> backfill -> enforce constraint -> remove legacy later.
- Never edit an already-applied migration in production history.
- Large backfills run in batches with monitoring.
- Enum changes require extra care; validated text + check constraints may be easier to evolve.

23. Backup and Recovery

- Automated daily backups plus point-in-time recovery if available.
- Backups encrypted and access-controlled.
- Restore tests validate schema, row counts, key queries, and application startup.
- Document RPO/RTO and retention schedule.
- Cache/materialized views are rebuildable and not primary backup targets.

24. Sample SQL / DDL

```
CREATE TABLE loans (  
  id uuid PRIMARY KEY,  
  reference_code varchar(24) NOT NULL UNIQUE,  
  borrower_id uuid NOT NULL REFERENCES users(id),  
  lender_id uuid NOT NULL REFERENCES users(id),  
  currency_code char(3),  
  principal_amount numeric(20,4),  
  interest_rate numeric(9,4),  
  interest_amount numeric(20,4),  
  expected_total numeric(20,4),  
  outstanding_amount numeric(20,4),  
  due_at timestamptz,  
  status varchar(32) NOT NULL,  
  accepted_at timestamptz,  
  completed_at timestamptz,  
  created_at timestamptz NOT NULL DEFAULT now(),  
  updated_at timestamptz NOT NULL DEFAULT now(),  
  row_version bigint NOT NULL DEFAULT 0,  
  CHECK (borrower_id <> lender_id),  
  CHECK (principal_amount IS NULL OR principal_amount > 0),  
  CHECK (outstanding_amount IS NULL OR outstanding_amount >= 0)  
);
```

Representative DDL is illustrative; exact enum/check definitions and migration syntax should match the selected ORM and deployment process.

25. Representative Queries

```
SELECT currency_code,  
       SUM(CASE WHEN lender_id = :user_id THEN outstanding_amount ELSE 0 END) AS receivable,  
       SUM(CASE WHEN borrower_id = :user_id THEN outstanding_amount ELSE 0 END) AS payable  
FROM loans  
WHERE (lender_id = :user_id OR borrower_id = :user_id)  
      AND status IN ('ACTIVE','OVERDUE','REPAYMENT_PENDING')  
GROUP BY currency_code;
```

The application derives $\text{net} = \text{receivable} - \text{payable}$ for each currency row. This query should be adapted if outstanding balance is derived rather than stored.

```
SELECT l.id, l.reference_code, l.due_at, l.outstanding_amount  
FROM loans l  
WHERE l.borrower_id = :user_id  
      AND l.outstanding_amount > 0  
      AND l.due_at < now()  
ORDER BY l.due_at ASC;
```

26. Data Reconciliation and Auditing

- Nightly/periodic job recomputes confirmed repayment totals by loan and compares with denormalized confirmed_repaid_amount/outstanding_amount if those fields are stored.
- Mismatch is logged and alerted; automatic correction should be conservative and audited.
- Check for impossible states: COMPLETED with outstanding > 0, ACTIVE with completed_at set, bank share owner mismatch, repayment confirmer not equal to lender.
- Foreign-key and orphan checks are part of migration/maintenance diagnostics.

27. Data Dictionary Summary

Entity	Authoritative Meaning
users	Stable account identity and profile lifecycle.
friendships	Connection/block relationship between two users.
loans	Current shared loan state and accepted headline financial fields.
loan_terms	Versioned historical/proposed terms.
repayments	Borrower repayment claims and lender confirmations.
bank_profiles	Owner-managed encrypted repayment destinations.
bank_profile_shares	Explicit authorization to reveal/use a bank profile.
loan_events	Append-only financial/history timeline.
notifications	User-facing in-app/push notification record.
device_tokens	Push delivery endpoints.
idempotency_records	Replay protection for state-changing API actions.
attachments	References to private files stored outside the DB.

28. Future Schema Extensions

Extension	Schema Preparation
Partial repayments	repayments already store arbitrary amount and multiple rows per loan.
Installments	Add loan_installments schedule table and allocation table.
APR/compounding	Add calculation_method + rate_period + schedule parameters; retain old term versions.
Payment processor	Add payment_accounts/token references and payment_transactions; never repurpose raw bank profile rows.
Disputes	Add disputes and dispute_messages/evidence tables with audited resolution.
FX estimates	Add fx_rate_snapshots with source/time; keep original-currency amounts immutable.
Loan agreement PDF	Add generated_documents table referencing accepted term version and content hash.
Organizations/family groups	Add groups/memberships and carefully scope visibility; do not weaken pairwise authorization by default.